

1 Les listes

Une liste est une structure de données qui permet de stocker plusieurs valeurs mais de type hétérogène, simple où composé.

La table 1 regroupe les méthodes de création d’une liste, la table 2 regroupe les opérations qu’une liste supporte et la table 2 les méthodes dont elle dispose.

Opération	Description
<code>l = []</code> ou <code>l = list()</code>	Crée un liste vide
<code>l = [1, 2, "CPGE"]</code>	Crée une liste initialisée par 3 éléments
<code>l = list(range(n))</code>	Crée une liste contenant les éléments de 0 à n-1
<code>l=[f(i) for i in range(n)]</code>	Crée une liste contenant les images de i par la fonction f avec i varie de 0 à n-1

TABLE 1 – Création d’une liste

Opération	Description
<code>x in l</code>	<i>True</i> si x est un des éléments de l
<code>x not in l</code>	Réciproque de la ligne précédente.
<code>l + t</code>	Concaténation de l et t .
<code>l * n</code>	Concatène n copies de l les unes à la suite des autres.
<code>l[i]</code>	Retourne le i^{eme} élément de l .
<code>l[i : j]</code>	Retourne une liste contenant les éléments de l d’indices i à j exclu.
<code>l[i : j : k]</code>	Retourne une liste contenant les éléments de l dont les indices sont compris entre i et j exclu, ces indices sont espacés de k : $i, i + k, i + 2k, i + 3k, \dots$
<code>len(l)</code>	Retourne le nombre d’éléments de l .
<code>min(l)</code>	Retourne le plus petit élément de l .
<code>max(l)</code>	Retourne le plus grand élément de l .
<code>sum(l)</code>	Retourne la somme de tous les éléments de l .
<code>del(l[i : j])</code>	Supprime les éléments d’indices entre i et j exclu. Cette instruction est équivalente à <code>l[i : j] = []</code> .
<code>list(x)</code>	Convertit x en une liste quand cela est possible.

TABLE 2 – Opérations disponibles sur les listes, on suppose que l et t sont des listes, i et j sont des entiers. x est quant à lui quelconque.

Méthode	Description
<i>l.append(x)</i>	Ajoute <i>x</i> à la fin de <i>l</i> .
<i>l.index(x)</i>	Retourne l'indice de la première occurrence de <i>x</i> dans <i>l</i> .Si celle-ci n'existe, une exception est déclenchée.
<i>l.count(x)</i>	Retourne le nombre d'occurrences de l'élément <i>x</i> dans <i>l</i> .
<i>l.extend(t)</i>	Ajoute tous les éléments de la liste <i>t</i> à la liste <i>l</i> (équivalent à <i>l+= t</i>).
<i>l.insert(i,x)</i>	Insère l'élément <i>x</i> à la position <i>i</i> dans la liste <i>l</i> .
<i>l.remove(x)</i>	Supprime la première occurrence de l'élément <i>x</i> dans la liste <i>l</i> . S'il n'y a aucune occurrence de <i>x</i> , cette méthode déclenche une exception.
<i>l.pop(i)</i>	Retourne l'élément <i>l[i]</i> et le supprime de la liste. Le paramètre <i>i</i> est facultatif, s'il n'est pas précisé, c'est le dernier élément dont la valeur est d'abord retournée puis il est supprimé de la liste.
<i>l.reverse()</i>	Renverser la liste.
<i>l.sort()</i>	Trier la liste dans l'ordre croissant.

TABLE 3 – Méthodes permettant de modifier une liste, on suppose que *l* et *t* sont des listes, *i* est un entier. *x* est quant à lui quelconque.


Attention !

L'ajout d'un élément à la liste s'effectué seulement par les méthodes déjà citées. On ne peut pas affecter à une case non existante une valeur ! c'est-à-dire $l[i] = valeur$ n'est correcte que si la case *i* existe déjà dans *l* !

Parcours d'une liste

Afficher tous les éléments impairs de la liste L en complétant le code.

```

1 L = [3, 7, 2, 9, 4, 19, 34, 445, 1, 13]
2 for ..... in .....:
3     if .....:
4         print(.....)
5

```

Exercice 2 : Parcours d'une liste

Afficher le carré de chaque élément de la liste L en complétant le code.

```

1 L = [3, 7, 2, 9, 4]
2 for ..... in .....:
3     print(.....)

```

Création d'une liste

Écrire une fonction `creer_liste(n)` qui :

- prend en paramètre un entier *n* ;
- crée une liste contenant les entiers de 1 à *n* ;
- retourne la liste L.

```

1 def creer_liste(n):
2     L = .....

```

```

3     for .... in .....:
4         L.append(.....)
5     return .....
6
7 n = int(input("Donner n : "))
8 print("Liste =", creer_liste(n))
9

```

Somme des éléments

Écrire une fonction `somme_liste(n)` qui :

- prend en paramètre un entier n ;
- crée une liste contenant les entiers de 1 à n ;
- retourne la somme des éléments de la liste.

```

1 def somme_liste(n):
2     L = .....#Créer une liste de n éléments
3     s = 0
4     for .... in .....:
5         s = .....
6     return .....
7
8 n = int(input("Donner n : "))
9 print("Somme =", somme_liste(n))

```

Multiplication scalaire d'une liste

Écrire une fonction Python `smul` à deux paramètres un nombre réel ou entier a et une liste L de nombres. La fonction doit multiplier chaque élément de la liste par le nombre a et renvoyer une nouvelle liste. `smul(2, [1, 2, 3]) → [2, 4, 6]`

```

1 def smul(a, L):
2     L2 = []
3     for .... in .....:
4         .....
5     return .....
6
7 print(smul(2, [1, 2, 3]))
8

```

Somme terme à terme de deux listes

Écrire une fonction Python `vsom` qui prend en paramètre deux listes de nombres $L1$ et $L2$ de même longueur, et qui renvoie une nouvelle liste contenant la somme terme à terme des deux listes.

`vsom([1, 2, 3], [4, 5, 6]) → [5, 7, 9]`

```

1 def vsom(L1, L2):
2     L3 = []
3     for ..... in .....:

```

```
4         .....
5     return .....
6
7 print(vsom([1, 2, 3], [4, 5, 6]))
8
```

Recherche séquentielle Sans utiliser les fonctions prédéfinies `index()`, `count()`....

La recherche séquentielle d'un élément dans une liste consiste à parcourir la liste séquentiellement jusqu'à trouver l'élément recherché ou atteindre la fin du tableau.
La recherche peut se faire sur une liste triée ou non, la différence réside dans le parcours de la liste et dans la condition d'arrêt de la recherche.

- ❶ Écrivez une fonction **rechSeq1(L,x)** qui effectue une recherche séquentielle de x (entier) parmi les n éléments d'une liste L et qui renvoie l'indice de sa première occurrence.
- ❷ Écrivez une fonction **rechOccurence(L,x,r)** qui renvoie *True* si l'entier x apparaît exactement r fois dans L et *False* sinon.

Palindrome

Écrire une fonction **Palindrome(L)** qui prend en paramètres une liste de caractères L . La fonction renvoie *True* si la liste est un palindrome et *False* sinon.
Proposer une solution itérative et une solution récursive.



Un palindrome est un mot qui peut être lu des deux sens :
Exemple L : ["K","a","y","a","K"]...

Plus grand élément

Écrire une fonction qui renvoie le plus grand élément d'une liste d'entiers (L'utilisation de la fonction **max()** n'est pas permise). On supposera que la liste est de longueur au moins 1 ; en revanche, on veillera à ne le parcourir qu'une seule fois.

Les deux plus grands éléments

Écrire une fonction qui renvoie les deux plus grands éléments d'une liste d'entiers. On supposera que la liste est de longueur au moins 2 ; en revanche, on veillera à ne le parcourir qu'une seule fois.